

IoT Interface For Real-time Voltage Mode Pulse Oximetry

Biplov Pokhrel (119EC1010)



Department of Electronics and Communication
Engineering

National Institute of Technology Rourkela

IoT Interface For Real-time Voltage Mode Pulse Oximetry

*Thesis submitted in partial
fulfillment of the requirements of
the degree of*

Bachelor of Technology

In

Electronics and Communication Engineering

By

Biplov Pokhrel

(Roll Number: 119EC1010)

*based on research
carried out under the
guidance of*

Dr. Sougata Kumar Kar



National Institute of Technology Rourkela

Department of Electronics & Communication Engineering



**Department of Electronics and Communication
Engineering National Institute of Technology Rourkela**

May 11, 2023

Certificate of Examination

Roll Number: 119EC1010

Name: BIPLOV POKHREL

Title of Dissertation: IoT Interface for Real-time Voltage Mode Pulse Oximetry

We, the below signed, after checking the thesis mentioned above and the official record book (s) of the student, hereby state our approval of the thesis submitted in partial fulfillment of the requirements of the degree of *Bachelor of Technology in Electronics and Communication Engineering* at *National Institute of Technology Rourkela*. We are satisfied with the volume, quality, correctness, and originality of the work.

Dr. Sougata Kumar Kar
Supervisor



**Department of Electronics and Communication
Engineering National Institute of Technology Rourkela**

Dr. Sougata Kumar Kar

Associate Professor

May 11, 2023

Supervisor's Certificate

This is to certify that the work presented in the thesis entitled “IoT Interface for Real-time Voltage Mode Pulse Oximetry” submitted by **Biplov Pokhrel**, Roll Number 119EC1010, is a record of original research carried out by him under my supervision and guidance in partial fulfillment of the requirements of the degree of **Bachelor of Technology** in **Electronics and Communication Engineering**. Neither this thesis nor any part of it has been submitted earlier for any degree or diploma to any institute or university in India or abroad.

Dr. Sougata Kumar Kar

Supervisor

Dedication

To my parents

And

my dear friends...

Declaration of Originality

I, Biplov Pokhrel Roll Number *119EC1010* hereby declare that this thesis entitled “IoT Interface for Real-time Voltage Mode Pulse Oximetry” represents my original research done by me, for the pursuit of Bachelor of Technology degree from National Institute of Technology, Rourkela. To the best of my knowledge, I have not used any material or work published by another person, in the pursuit of their degree or diploma from any institution or university. I have already acknowledged the contributions of my supervisor and other professors in the Acknowledgement section, and wherever I have cited works of other authors, I have duly mentioned them under “Reference Section”.

May 11, 2023
NIT Rourkela

Biplov Pokhrel
119EC1010

Acknowledgement

I am deeply grateful to all the individuals who have played a pivotal role in making this project a reality. Their unwavering support, encouragement, and guidance have been instrumental in enabling me to complete this project successfully.

Foremost, I am profoundly grateful for the immense support and guidance extended to me by my supervisor, **Dr. Sougata Kumar Kar**, throughout the course of this research. His unwavering commitment to excellence and exceptional expertise in the field of Internet of Things have been a tremendous source of inspiration and motivation for me. His insightful feedback, constructive criticism, and invaluable inputs have been instrumental in shaping my work.

I would also like to extend my thanks to my colleagues, whose thought-provoking discussions and insightful inputs have been a constant source of inspiration to me. Their valuable feedback and constructive criticism have been instrumental in shaping my work and enhancing its quality.

May 11, 2023

NIT Rourkela

Biplov Pokhrel

119EC1010

Abstract

In this project, an IoT interface for an existing system of a non-invasive and low-power transimpedance amplifier-based pulse oximeter that accurately measured SpO₂ and heart-rate was created. The amplifier converted the photocurrent generated by the photodetector into voltage, resulting in the PPG signal. The system operated within ranges of 70 – 100% for SpO₂ and 25 – 300 bpm for heart-rate. The measured values were displayed on a 3.5-inch TFT LCD, and simultaneously sent to a Firebase database in real-time, allowing users to monitor their vital signs on their mobile or desktop devices. A user-friendly React-JS website was developed and linked to Firebase, providing a real-time display of the PPG waveform and measured values with a sampling rate of 10 samples/s. Optimizations were done to send the data value to the website in the fastest allowable rate. The proposed pulse oximeter system along with its IoT interface provides a reliable and accurate way to monitor vital signs and can assist in making informed decisions about health.

Keywords

Pulse Oximetry, Health Monitoring, Trans-impedance Amplifier, Internet of Things, FERN stack

Contents

Certificate of Examination	iii
Supervisor's Certificate	iv
Dedication	v
Declaration of Originality	vi
Acknowledgement	vii
Abstract	viii
List of Figures	xi
CHAPTER 1	1
Introduction	1
1.1 Motivation	2
1.2 Problem Statement	3
1.3 Literature Review	4
1.4 Thesis Arrangement	5
CHAPTER 2	7
Pulse Oximetry	7
2.1 Pulse Oximetry and Oxygen Saturation	7
2.2 Working Principle	8
2.3 PPG Analysis	9
2.4 Beer-Lambert's Law	10
2.5 Limitations of Pulse Oximetry	12
CHAPTER 3	13
Hardware Architecture	13
3.1 LED Driver Circuit	13
3.2 Transimpedance Amplifier and Filter Blocks	15
3.3 Interfacing the LCD Display and ESP Module	17
CHAPTER 4	18
SpO2 and Heart Rate Determination	18
4.1 Peak-Valley Method	18
4.2 AC and DC Separation Method	19
4.3 Heart Rate Determination	19
4.4 Calculated Values and Comparisons	20
CHAPTER 5	22
Software Architecture	22
5.1 Arduino Setup	22
5.2 Database Setup	27

5.3	Website Setup	28
5.4	Code Optimization for Sampling Rates	31
5.5	Program Flow	35
CHAPTER 6		36
Results and Conclusion		36
6.1	Results	36
6.2	Conclusion	38
6.3	Scope for future works	38
References		39

List of Figures

Figure 2. 1 Absorption Coefficient of Hb, HbO ₂ , H ₂ O	9
Figure 3. 1 Architecture of the proposed oximeter system	14
Figure 3. 2 Nellcor Probe Pinout Diagram	14
Figure 3. 3 Schematic for LED Driver Circuit	14
Figure 3. 4 Schematic of Transimpedance Amplifier Circuit	15
Figure 3. 5 Transimpedance Amplifier Output in accordance to signals provided	15
Figure 3. 6 Block Diagram for interfacing between Nellcor probe, Esp8266 and Arduino Mega	16
Figure 4. 1 Output of Transimpedance amplifier circuitry and input to Arduino Mega	18
Figure 4. 2 Filtered Signals for Red and IR	20
Figure 4. 3 Separated AC and DC Components of Red and IR	21
Figure 5. 1 Arduino Setup Block Diagram	22
Figure 5. 2 LED driver logic flow chart	23
Figure 5. 3 Data being sent to the ESP Module from Arduino Mega	25
Figure 5. 4 Receiving, Processing and Sending data within ESP Module	26
Figure 5. 5 Realtime database overview	28
Figure 5. 6 React project overview	29
Figure 5. 7 Patient Entry Form	31
Figure 5. 8 Program and Data flow	35
Figure 6. 1 PCB Prototype of the System	36
Figure 6. 2 Graph and Health Indicators on local display	37
Figure 6. 3 Website UI for graph and patient details	37

CHAPTER 1

Introduction

Oxygen is an essential element for human life and a lack of oxygen can lead to various symptoms such as dizziness, headaches, and eventually breathing problems or even blackout. The COVID-19 pandemic has made people more aware of their oxygen levels, measured as saturation percentage. Among various methods like chemical-based, electrode-based, transducer-based, and spectrophotometer-based methods, the spectrophotometer method is more preferred due to its non-invasive, fast, continuous and accurate results. This method involves passing two specific wavelengths of light (660nm and 900nm) through a finger and measuring their intensities at the other end. The absorptivity or extinction coefficient of the substance, the path-length over the material, and the concentration of the substance all affect the intensity absorption at certain wavelengths. By analyzing the variation in intensities of both lights, the SpO₂ (oxygen saturation percentage) can be determined.

For the human body to produce energy, oxygen is one of the crucial elements required. In the presence of oxygen, Glucose is oxidized to form CO₂, H₂O and ATP. Hemoglobin (Hb) is the primary means of delivering oxygen to the tissues of the body and it performs the best among other oxygen carriers. Hb carrying oxygen is referred to as Oxygenated Hemoglobin (HbO₂) and the Hb not carrying oxygen is referred to as Deoxygenated (Hb). Thus, the determination of these two types of Hbs' concentration is important so as to find the amount of oxygen present in the Hemoglobins.

Hemoglobins play the important role of delivering oxygen to tissues of the body and their malfunction can lead to severe health hazard and consequences. Thus, finding concentrations of HbO₂ and Hb to find the amount of oxygen and monitoring oxygen levels by this means can help manage conditions like anemia, lung and heart diseases, etc. The spectrophotometer method is widely used for measuring oxygen saturation levels as it provides quick, continuous, and more accurate and precise results. This method involves the measurement of the absorption of light of specific wavelengths passed through the finger to determine the intensity of HbO₂ and deoxygenated Hb. The SpO₂ levels are then determined by analyzing the variation in the intensity of both lights.

For the body to operate properly, ideal oxygen saturation levels must be maintained.

Hypoxemia, which can cause a variety of symptoms like shortness of breath, a rapid heartbeat, and confusion, can be brought on by low levels of oxygen saturation. Long-term hypoxemia can harm crucial organs like the heart, brain, and lungs. On the other hand, excessive oxygen saturation can harm cells and tissues through oxidative stress. Consequently, it is crucial to regularly check oxygen saturation levels in order to avoid any serious repercussions and maintain general health.

Apart from oxygen-carrying hemoglobin (HbO₂), there are other types of dysfunctional hemoglobin present in the human body, including Methemoglobin (MethHb), Carboxyhemoglobin (COHb), Sulfhemoglobin (SHb), and Carboxysulfhemoglobin (COSHb). MethHb is formed through oxidation of hemoglobin and loses its ability to bind with oxygen, while COHb is formed when Hb comes in contact with carbon monoxide and has a higher affinity for CO than for oxygen. SHb is formed by the combination of Hb and hydrogen sulfide and has significantly less oxygen-carrying capacity compared to HbO₂. Fractional hemoglobin saturation, the ratio of HbO₂ to the total hemoglobin (including all types of Hbs), is a significant term in determining oxygen saturation levels in the body.

Fractional SaO₂ and SaO₂ both describe oxygen saturation levels in the body, but there is a difference between the two. Fractional SaO₂ is the ratio of oxygenated hemoglobin to the total hemoglobin present in the body, including dysfunctional hemoglobins. SaO₂, on the other hand, is the ratio of oxygenated hemoglobin to the total hemoglobin of only functional hemoglobins and is measured using pulse oximetry.

1.1 Motivation

Analyzing data such as oxygen saturation levels in real-time can have significant benefits for patient outcomes. By quickly detecting changes in oxygen saturation levels, healthcare professionals can prevent serious complications such as headaches, shortness of breath, heart malfunctions, and brain malfunctions. Various traditional methods like chemical method, electrode method, and spectrophotometer-based methods can be used to detect blood oxygen levels; where pulse oximetry through spectrophotometry is the best non-invasive method available.[1] Pulse oximetry, which is commonly used in various hospital departments, including anesthesiology, can provide valuable information to healthcare professionals to ensure that patients are receiving the appropriate treatment.

Pulse oximetry is a widely used non-invasive technique for monitoring oxygen saturation levels in the blood. It is used in almost all sections of hospital departments, including anesthesiology, where it is crucial for monitoring patients during surgeries. However, analyzing the data produced by pulse oximeters and other monitoring devices can be time-consuming and difficult for healthcare professionals.

To address this issue, the development of a system that allows for remote observation of patients and easy maintenance of data is critical. By using a web-based platform that collects and displays patient data in real-time, healthcare professionals can easily monitor and analyze patient health indicators from anywhere with an internet connection.

Our proposed system will utilize a React website and a Firebase database to store and display patient health indicator data in real-time. This system will enable healthcare professionals to remotely monitor patients' health indicators and receive alerts in case of any abnormal readings. Additionally, the system will simplify maintenance by automating data collection and reducing the need for physical visits to collect patient data. The proposed system aims to provide a reliable and efficient method for remotely monitoring patients' health indicators while minimizing maintenance costs and efforts. The development of such a system will have significant implications for improving patient care and reducing healthcare costs.

1.2 Problem Statement

The problem statement revolves around the challenge of monitoring and analyzing patient health indicators such as oxygen saturation levels in real-time, which is crucial for detecting any potential complications and ensuring patients receive appropriate treatment. Although pulse oximetry and other monitoring devices are commonly used in hospital departments, analyzing the data produced by these devices can be time-consuming and challenging for healthcare professionals. Therefore, there is a need for a system that allows for remote observation of patients and easy maintenance of data to simplify the monitoring process. The proposed system will use a React website and a Firebase database to store and display patient health indicator data in real-time, enabling healthcare professionals to remotely monitor patients' health indicators and receive alerts in case of any abnormal readings. The system will simplify maintenance by automating data collection and reducing the need for physical visits to collect patient data. The proposed system aims to provide a reliable and efficient method for remotely monitoring patients' health indicators while minimizing maintenance costs and efforts, which will have significant implications for improving patient care.

1.3 Literature Review

The field of pulse oximetry has a long history spanning over 80 years, with constant advancements in technology. The journey began in 1939 when Karl Matthes from Leipzig introduced ear oximetry, which involved using red and infrared light to measure intensities at the other end of the earlobe.

In 1940, J. R. Squire [2] discovered that there were differences in transmission between red and infrared light, and the variation in the intensity of these lights across the earlobe was an indicator of oxygen saturation. G. A. Millikan [3] also created a functional earlobe pulse oximeter in the same year. However, the disadvantage of their design was the need for zero-point calibration, which required squeezing the earlobe for a specific duration to stop circulation in the arteries temporarily. Once released, the oxygen saturation level could be measured. Millikan was also the first person to name this type of device an "oximeter."

In 1949, Earl Wood [4] and J.E. Geraci modified Millikan's earpiece design by incorporating Squire's theory and developed a mathematically robust concept for plotting the ratio of red to infrared light optical ear density as a function of oxygen saturation without requiring zero set-point calibration. In the 1970s, Hewlett Packard (HP) Corporation developed an ear oximeter that used eight different wavelengths without requiring external calibration. The device analyzed arterial blood by heating the earlobe and using the eight equations of Beer-Lambert's law to obtain eight unknown concentrations of HbO₂, Hb, MetHb, and also absorption due to skin, tissues, and bones. This oximeter had greater accuracy than Millikan's oximeter, but was more complex and expensive. Overall, these advancements in technology have led to improved accuracy and convenience in measuring oxygen saturation levels.

In 1974, Takuo Aoyagi [5] discovered that an ear oximeter could record a dye dilution curve, but it required calibration with a blood sample. He used a 630-nm wavelength, which was most sensitive to oxygen saturation, and a 900-nm infrared wavelength, which was not absorbed by dye, to balance the measurements. A green dye was also used to measure cardiac output. Aoyagi on inspection discovered that there was a decrease with desaturation in the optical density of blood at 900-nm. It resulted in a signal that was larger than provided by 600-nm and 805-nm ratios.

In recent years, there have been significant advancements in pulse oximetry technology. For example, in 2010, M. Tavakoli [6] developed an ultra-low power current mode pulse oximeter that uses only 4.8 mW of total power to operate and can run for 60 days on just 4-AAA batteries. He achieved this by redesigning the traditional transimpedance amplifier into a novel

logarithmic transimpedance amplifier. In 2013, The first fully integrated pulse oximeter system with the consumption of power up to a maximum of 1mW was developed by K. N. Glaros [7]. The entire chip was fabricated on the AMS 0.35 μm technology and occupies a total area of 1.35 mm².

Another significant development was the reflective type pulse oximeter developed by A. Azhari in 2017 [8]. Weighing only 15 g, this oximeter can be easily worn on the forehead and is integrated with wireless connectivity to digitize and transmit data via a Bluetooth module in real-time. The SpO₂ values are then calculated on a remote PC.

Now, with the rise of the Internet of Things (IoT), research in pulse oximetry is increasingly focused on incorporating IoT technology. In 2021, A. Chatterjee developed an IoT-based pulse oximeter system that can monitor a patient's heartbeat and oxygen saturation level from any part of the hospital premises [9]. Heart rate and SpO₂ are measured in real-time and can be displayed on Android smartphones, allowing medical staff to monitor patients remotely and respond quickly in case of emergency.

Continuing these research and development, we will work on this paper on building a website that will not only show the health indicators in real-time but also the PPG and ECG graphs in real-time, as well as provide a database to store all the patient's history in great detail for further study.

1.4 Thesis Arrangement

Chapter 1 of this thesis provides a brief introduction to pulse oximetry and the motivation for working in this domain. It discusses the importance of pulse oximetry in modern healthcare and highlights the remarkable advancements in the field over the past few decades. This chapter serves as a foundation for the following chapters, providing a context for the work done in this thesis.

Chapter 2 delves into the basics of pulse oximetry and provides an in-depth analysis of oxygen saturation. The principle of pulse oximetry is discussed, along with the analysis of photoplethysmography (PPG). Theoretical determination of SpO₂ is done, and the limitations of pulse oximetry are also discussed along with their possible solutions.

Chapter 3 elaborates on the proposed architecture for the real-time voltage mode pulse oximeter. It explains each component of the architecture in detail and provides a comprehensive understanding of how they work together to create an efficient and accurate oximeter.

In **Chapter 4**, the algorithms for determining R and heart-rate are explained with the help of real-time instances. This chapter focuses on the technical aspects of the implementation and provides an insight into the complex calculations involved in determining accurate readings. It also showcases the results of the implementation and their significance. This chapter provides a detailed analysis of the accuracy and efficiency of the proposed pulse oximeter, highlighting its strengths and limitations.

In **Chapter 5**, we discuss about setting up the database, the website, and the code implementations with all the optimization techniques on the devices so as to get considerable sampling rate. A working website to show real-time data is created in this chapter.

Finally, **Chapter 6** concludes the thesis by summarizing the work done, the results and the future scope of this field.

CHAPTER 2

Pulse Oximetry

2.1 Pulse Oximetry and Oxygen Saturation

Pulse oximetry is a method used to measure SpO₂ levels in the body. To understand this method, it's important to know how oxygen is distributed in the body and what needs to be measured to determine the concentration of oxygen in the body. Oxygen is transported from the lungs to different parts of the body through hemoglobin (Hb) or plasma. Hb is more efficient than plasma, as it contains the heme molecule. By measuring the concentrations of these molecules, we can determine the amount of oxygen in the body. The relationship between the amount of oxygen carried by Hb and plasma is given by a formula:[1]

$$C_{HbO_2} = 1.37 * Hb * SaO_2 \quad (2.1)$$

$$C_{DO_2} = 0.003 * PaO_2 \quad (2.2)$$

cHbO₂ and cDO₂ represent the amount of oxygen carried by hemoglobin and plasma, respectively, in 100 mL of blood. SaO₂ is the arterial oxygen saturation, while P aO₂ is the partial pressure of oxygen. The constant 1.37 represents the number of mL of oxygen bound to 1 g of fully saturated Hb. The total volume of oxygen in the body can be determined by adding the contributions of hemoglobin and plasma, but the contribution of plasma is minimal compared to that of hemoglobin. Hence, the total volume of oxygen can be determined by considering only the hemoglobin part.

The SaO₂ can be determined to measure the amount of oxygen in the body, and this is typically done through pulse oximetry. Pulse oximetry is a non-invasive method that uses a spectrophotometric technique to measure peripheral oxygen saturation or SpO₂. This method has an accuracy of less than 3% for oxygen saturation levels greater than 70% [1], but the accuracy decreases as the oxygen saturation increases. Pulse oximetry is widely used in the medical field due to its fast and continuous measurement of SpO₂ values, non-invasive nature, and ease of use. It can also be integrated with other signals like ECG for further analysis and advancement in technology.

Oxygen saturation (SpO₂) is critical for a person's health. Hypoxemia, a condition where SpO₂ is lower than the desired level (generally 95% to 100%), can cause direct effects such as dizziness, headache, and more serious indirect effects such as hypoxia. Hypoxia refers to a condition where there is a lower amount of O₂ in tissues, which can also occur due to low oxygen delivery index (DIO₂) even if there is ample O₂ in the blood.

$$D_I O_2 = 10 * CaO_2 * CI \quad (2.3)$$

DIO₂ represents the figure of merit for delivering O₂ from blood into tissues and is dependent on the cardiac index (CI), which is the amount of blood pumped by the left ventricle in liters per minute, and CaO₂, the total volume of O₂ per 100 ml of blood. Conversely, higher than normal levels of O₂ can be harmful due to the toxic nature of O₂ radicals, which can cause hyperoxemia in the blood and hyperoxia in tissues.

2.2 Working Principle

Studies have shown that hemoglobin (Hb) emits a red glow when oxidized by oxygen. When Hb changes from a dark red to a bright red color due to oxygenation, its absorption of red-light is minimized, resulting in more red light being emitted by HbO₂ when red light is shone on it. By transmitting red light across the finger and receiving the rest of the red light on the opposite side with a photo-detector, the SpO₂ value can be determined. To reduce the dependence on other factors, infrared (IR) light is used on top to calculate absorptions, and the ratio of the intensities of both red and IR light gives the SpO₂ value. The choice of wavelengths is made based on the absorption coefficient difference, which must be sufficiently large and flat over a small range of adjacent wavelengths in order to minimize wavelength generation mistakes. As a result, the wavelength of 660nm (RED) and 900nm (IR) is used. The variation in absorption coefficients of Hb and HbO₂ as regards to wavelength is depicted in Fig. 2.1. By taking the ratio of the two absorptions, the SpO₂ value becomes more independent of other factors, as explained in the following section.

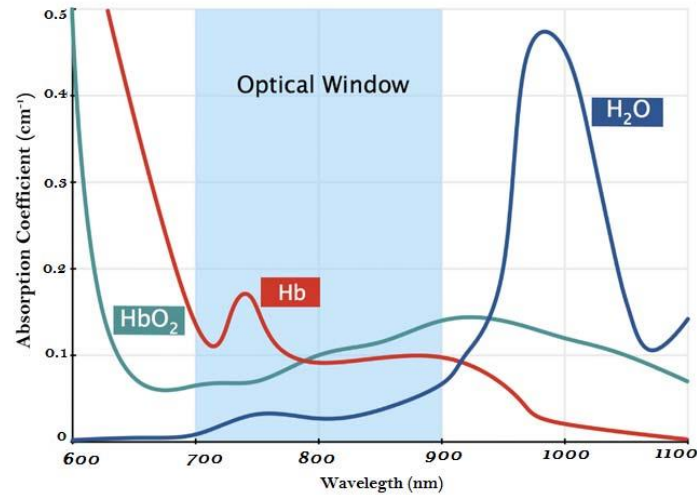


Figure 2. 1 Absorption Coefficient of Hb, HbO₂, H₂O [10]

2.3 PPG Analysis

When RED or IR light is transmitted through a finger, it encounters several materials that cause absorption, including bones, tissues, skin, arterial blood, and venous blood. Arteries contain more blood during systole, when the heart pumps blood out, and less blood during diastole, when the heart is in a relaxed position. Therefore, during systole, arteries expand to accommodate the greater blood flow, while during diastole, arteries relax and return to their original thickness. Arteries exhibit variations in diameter, but veins do not. Due to these changes in diameter, the absorption of RED or IR light changes, and the intensity of the received output varies with time. This variation is referred to as photoplethysmogram (PPG). The PPG signal can help to determine the concentrations of Hb and HbO₂, which are required for SpO₂ calculations. It is also necessary to take into account difficulties such as variations in finger thickness during systole and diastole, the exponential dependence on the absolute concentration of hemoglobin in blood, and side effects like light scattering in human tissues and incoming light reflection.

2.4 Beer-Lambert's Law

First, Beer's law equation could be applied to the answers of these issues. There, the tissues and veins that don't pulse contribute to a constant absorbance that is denoted as the DC component, and the change brought on by the heartbeat cycle results in the variable AC component, which is path-length dependent. The equation of Beer's law is:

$$I = I_0 e^{-\epsilon lc} \quad (2.4)$$

Where I is the intensity sensed by the photoreceptor at the other end of the solution, I_0 is the initial intensity of incident light, ϵ is the net solution's molar absorption coefficient, l is the light's path length through the solution, and c is the solution's absolute concentration. Further, it can be represented in terms of transmittance then in terms of absorbance as:

$$T = \frac{I}{I_0} e^{-\epsilon lc} \quad (2.5)$$

$$A = -\ln(T) = \epsilon lc \quad (2.6)$$

The absorbance of the net substance is expressed as a summation of all individual absorbance of the substances in it and similarly, the total absorbance for the RED light is expressed as the sum of both HbO₂ and Hb, with their respective concentrations and path lengths of the light for them.

$$AT = \epsilon_1 l_1 c_1 + \epsilon_2 l_2 c_2 + \dots + \epsilon_n l_n c_n \quad (2.7)$$

$$AT = \epsilon_{HbO_2} l_{HbO_2} C_{HbO_2} + \epsilon_{Hb} l_{Hb} C_{Hb} \quad (2.8)$$

From the definition of SpO₂, HbO₂ and Hb are calculated as:

$$s_p O_2 = \frac{C_{HbO_2}}{C_{HbO_2} + C_{Hb}} \quad (2.9)$$

From 2.8 and 2.9, the AT will be:

$$A_T = [\epsilon_{HbO_2} s_p O_2 + \epsilon_{Hb} (1 - s_p O_2)] (C_{HbO_2} + C_{Hb}) d \quad (2.10)$$

It is also considered that the path length got both HbO₂ and Hb will be same, as they are being measured at the same instant. At diastole, arteries will be thin. Hence less absorption due to less path length, this will give the maximum intensity:

$$I_L = I_0 (e^{-\epsilon_{DC} c_{DC} d_{DC}}) \cdot (e^{-[\epsilon_{Hb} C_{Hb} + \epsilon_{HbO_2} C_{HbO_2}] d_{max}}) \quad (2.11)$$

At systole, arteries will be thick. Hence more absorption due to more path length, this will give the minimum intensity:

$$I_L = I_0(e^{-\epsilon_{DC}c_{DC}d_{DC}}).(e^{-[\epsilon_{Hb}c_{Hb}+\epsilon_{HbO_2}c_{HbO_2}]d_{max}}) \quad (2.12)$$

Therefore, the path length δd determines how much of an intensity variation is received at the opposite end. In terms of δd , the output intensity equation can be generalised as [1]:

$$I_H = I_0(e^{-[\epsilon_{Hb}c_{Hb}+\epsilon_{HbO_2}c_{HbO_2}]\delta d}) \quad (2.13)$$

Where δd will vary from d_{min} to d_{max} . Now, the general equation of intensity is still depended on initial light intensity and the path length. For reducing the dependency another wavelength other than RED (660nm) is used, which is IR (900nm). However, the initial intensity needs to be normalized because it's possible that IR LED intensity will differ from red LED intensity. The received maximum intensity will be divided by the initial raw intensity for normalization:

$$I_n = \frac{I}{I_H} = (e^{-[\epsilon_{Hb}c_{Hb}+\epsilon_{HbO_2}c_{HbO_2}]\delta d}) \quad (2.14)$$

$$R = \frac{A_{T,R}}{A_{T,IR}} = \frac{-\ln(T_R)}{-\ln(T_{IR})} = \frac{I_{n,R}}{I_{n,IR}} = \frac{V_{ac,R}/V_{dc,R}}{V_{ac,IR}/V_{dc,IR}} \quad (2.15)$$

$$R = \frac{[\epsilon(\lambda_R)_{Hb}c_{Hb}] - \epsilon(\lambda_R)_{HbO_2}c_{HbO_2}}{[\epsilon(\lambda_{IR})_{Hb}c_{Hb}] - \epsilon(\lambda_{IR})_{HbO_2}c_{HbO_2}} \quad (2.16)$$

Equation 2.16 can be re-framed in terms of SpO₂, which will give the final result as shown for SpO₂ in the following equations:

$$R = \frac{[\epsilon(\lambda_R)_{Hb} + (\epsilon(\lambda_R)_{HbO_2} - \epsilon(\lambda_R)_{Hb})SpO_2]}{[\epsilon(\lambda_{IR})_{Hb} + (\epsilon(\lambda_{IR})_{HbO_2} - \epsilon(\lambda_{IR})_{Hb})SpO_2]} \quad (2.17)$$

$$SpO_2 = \frac{\epsilon(\lambda_R)_{Hb} - \epsilon(\lambda_{IR})_{Hb}R}{[\epsilon(\lambda_R)_{Hb} - \epsilon(\lambda_{IR})_{HbO_2}] - [\epsilon(\lambda_{IR})_{HbO_2} - \epsilon(\lambda_{IR})_{Hb}]R} \quad (2.18)$$

The values of each of these constants are established as follows with the use of numerous lab records and experimentation iterations:

$$SpO_2 = \frac{0.81 - 0.18R}{0.63 + 0.11R} \quad (2.19)$$

2.5 Limitations of Pulse Oximetry

Pulse oximeters are used greatly in the medical field to measure the level of oxygen in the blood. While they have numerous benefits, there are also limitations to their use. A few of such limitations are mentioned below:

- Results can be inaccurate if the patient has poor circulation, is hypotensive, or has vasoconstriction.
- Some medical conditions, such as anemia or carbon monoxide poisoning, can cause inaccurate readings.
- The use of nail polish, skin pigmentation, or tattoos on the measurement site can interfere with accurate readings.
- False readings can occur if the probe is not placed correctly on the patient's skin, or if the device is malfunctioning or poorly calibrated.
- While pulse oximeters can provide a real-time reading of blood oxygen levels, they do not provide information on the overall oxygenation status of the body or the amount of oxygen being delivered to specific organs.

In summary, while pulse oximeters are a useful tool for measuring blood oxygen levels, their results should be interpreted in conjunction with other clinical information and used cautiously in certain patient populations.

CHAPTER 3

Hardware Architecture

The voltage mode pulse oximeter architecture is illustrated in Fig. 3.1, which uses an Arduino MEGA 2560 for timing, control, and voltage measurement. A trans-impedance amplifier is built with an op-amp of LM324N. To separate the IR and RED components, a HCF4051BE - 8 by 1 mux/demux IC is used. A 9V DC power supply is used to power the ICs and MEGA. The Nellcor probe is used, which has two LEDs (RED and IR) and a DB9 connector to interface the probe with the circuit. The Arduino MEGA is responsible for generating 5V pulses, compiling code, and sending digital output. The circuit's operation is described in more detail below.

3.1 LED Driver Circuit

Fig. 3.1 displays the schematic for the LED driver. The current passing through the LEDs is limited by the use of $470\text{ k}\Omega$ resistors. The current passing through the base of the transistor is adjusted by using $100\text{ k}\Omega$ resistors, which ensures a desired voltage is dropped at $470\text{ }\Omega$ resistors to prevent the LEDs from burning out. Pulses S0 and S1 are 100 Hz with a 20% duty cycle and are delayed by 3ms to allow only one LED to light up at a time. The current flow direction is shown in the diagram by the red and blue lines. When the input at the base of Q1 is high, Q1 will be ON and Q2 will be OFF, so the power supply voltage from the source is driven to D2 (RED LED) and D2 will be forward-biased (ON) with the shown current path. Conversely, when the input at the base of Q2 is high, Q2 will be ON and Q1 will be OFF, so the power supply voltage from the source is driven to D1 (IR LED) and D1 will be forward-biased (ON) with the shown current path. The output generated by this circuit drives the LEDs.

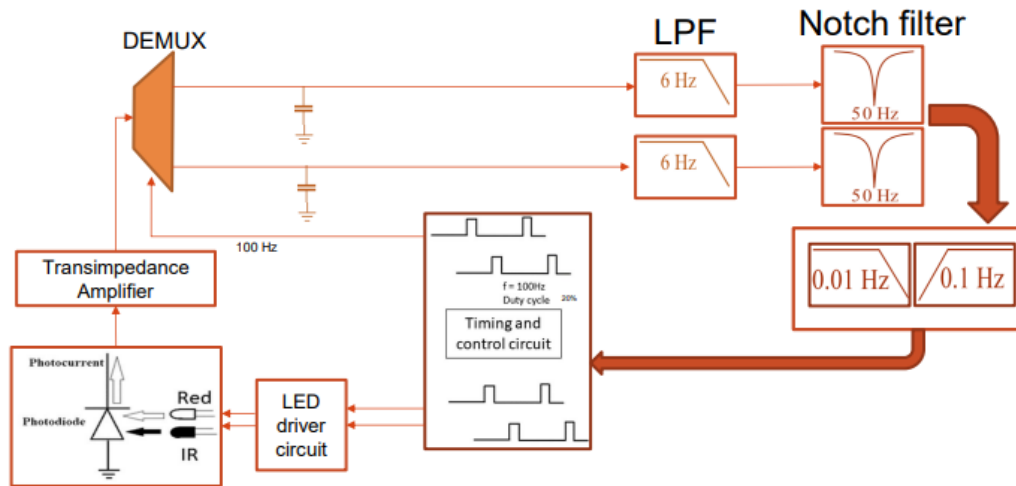


Figure 3. 1 Architecture of the proposed oximeter system

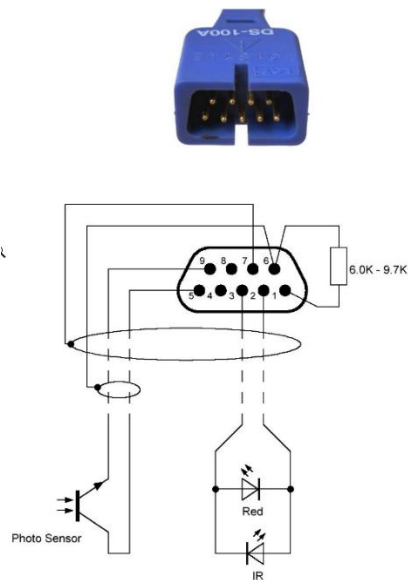


Figure 3. 2 Nellcor Probe Pinout Diagram

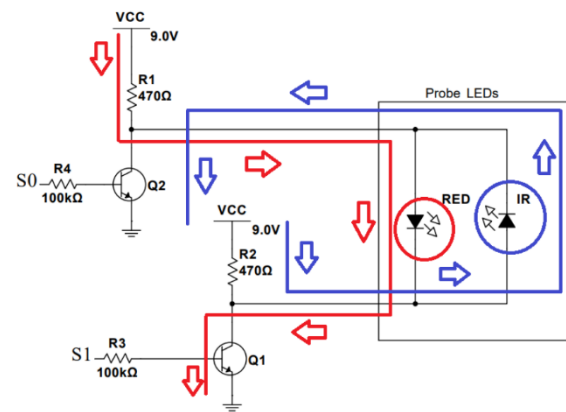


Figure 3. 3 Schematic for LED Driver Circuit

3.2 Transimpedance Amplifier and Filter Blocks

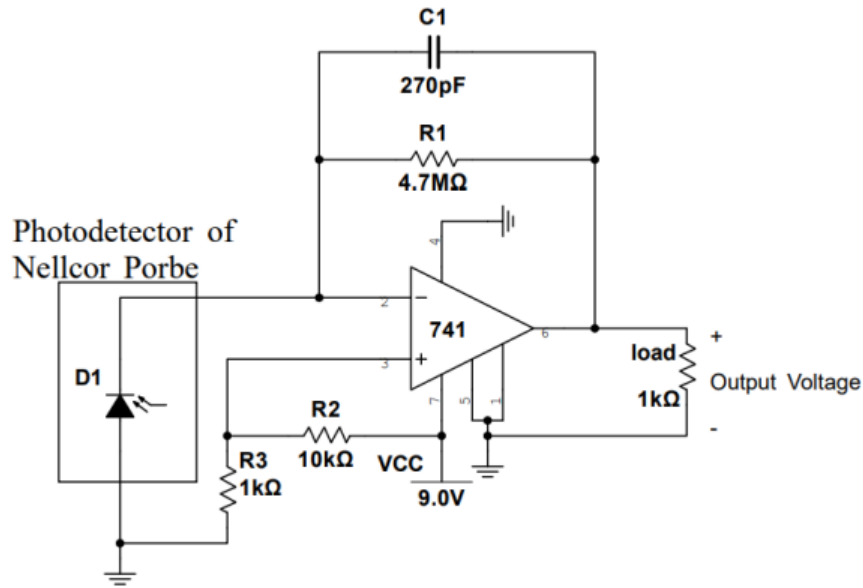


Figure 3. 2 Schematic of Transimpedance Amplifier Circuit

The above circuit is created where S0 and S1 are provided by the Arduino Mega such that they represent the following signal.

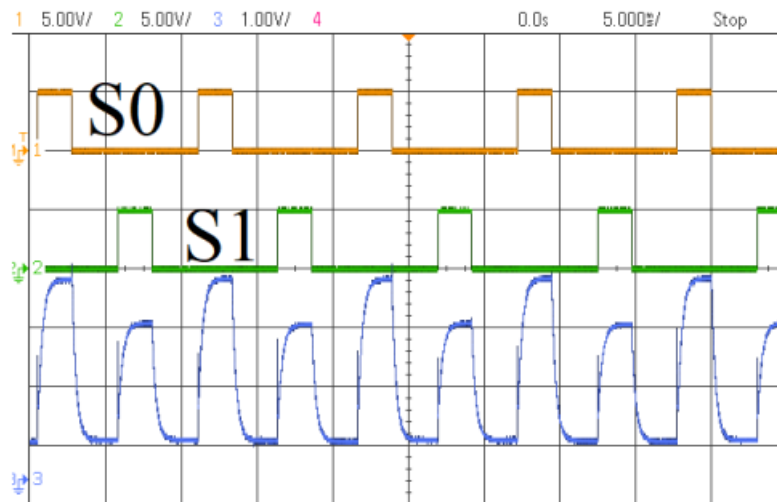


Figure 3. 3 Transimpedance Amplifier Output in Accordance to Signals Provided

The following block diagram represents the circuit created using the Arduino Mega and the Filter blocks are used to get AC and DC components of the IR and Red lights. The total circuitry and filtering process is explained below:

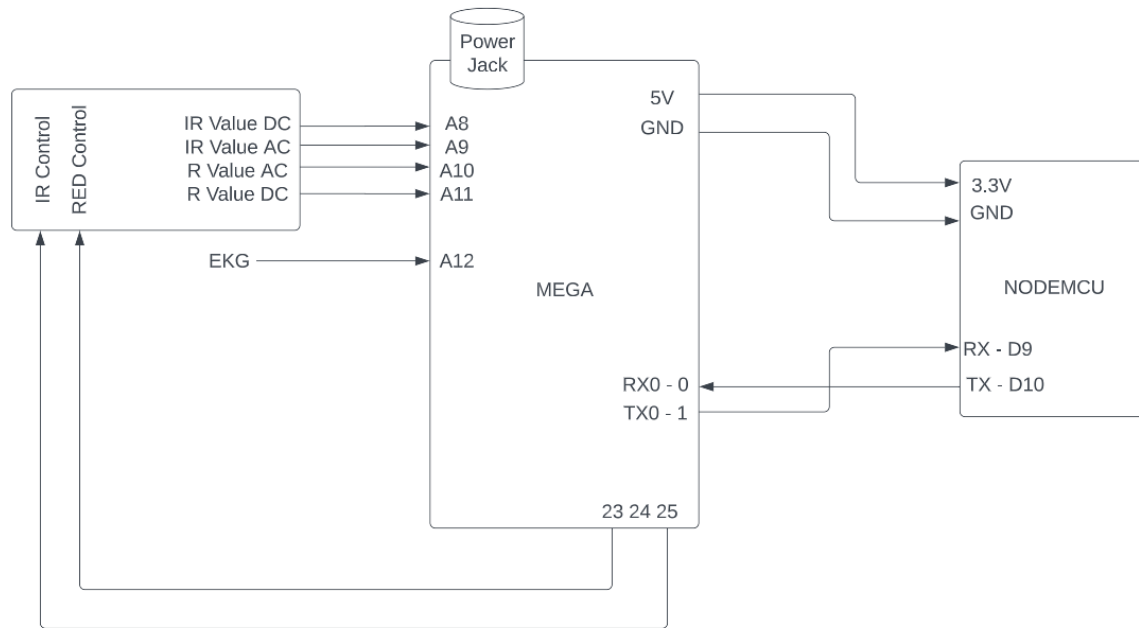


Figure 3. 4 Block Diagram for interfacing between Nellcor probe, Esp8266 and Arduino Mega

- The probe works in a configuration where the photo-sensor or photo-resistor captures one of the IR or Red light at a time. So, we need to create a circuit that drives the LEDs in a synchronous and alternate manner.
- We take out two digital outputs from the MEGA pins 23 and 25 and pair them up to the S0 and S1 signals as shown in the circuit below so as to drive the LEDs. s S0 and S1 are of 100 Hz with 20% duty cycle and have a delay of 3ms among them. This will play an important role in the sampling.
- Now talking about the signals. Pin 5 is the photoresistor output that is hooked up to the transimpedance amplifier to get the PPG followed by a circuit that separates IR and R voltages and their DC and AC components.
- This demodulator circuit makes use of the driving signals S1 and S0 to separate the contribution of the IR signal and R signal on the PPG graph.
- Passed through a 60 Hz line interference noise filter (Twin T notch) also used for AC-DC separation.
- Passed through a 6 Hz low pass filter because the PPG signal usually ranges between 0 to 5 Hz.
- Now we need to separate the AC and DC components that are used to find the R value that is used to find the SpO2.

- **Low Pass Filter (LPF):** The filtered signals are then passed through a single-order, 0.01 Hz LPF to extract the DC component of the signals. This filter removes any high-frequency AC components, leaving only the DC component, which represents the baseline signal.
- **High Pass Filter (HPF):** The filtered signals are then passed through a third-order HPF to extract the AC component of the signals. This filter removes the DC component, leaving only the AC component, which represents the signal variations caused by changes in blood volume in the microvasculature.
- **Output:** The output of the HPF is the AC component, while the output of the LPF is the DC component. These outputs represent the separated AC and DC components of the IR and RED signals.
- Now the 4 signals AC and DC of both IR and Red are supplied to the Arduino Mega where we take note of the time instances the signals are available to find the SpO2 values through various formulae. The signals are given to A8, A9, A10 and A11 pins on the Arduino Mega.
- Along with the Nellcor probe an EKG module too can be paired and its output fed to the A12 analog pin of the Arduino Mega to then process the data.

3.3 Interfacing the LCD Display and ESP Module

An Arduino shield with a 3.5-inch TFT LCD display is being utilized to showcase SpO2, heart rates, and both PPG and ECG waves in real-time. The shield type display is positioned on top of the Arduino MEGA without requiring any rewiring. It simply rests on the MEGA and can display the information.

To establish a power supply connection between the Arduino Mega and the ESP8266, the 5V power output from the former is linked to the 3.3V power input on the latter. Additionally, the ground wires on both devices are connected. For data transmission, the serial printer on the Arduino Mega is utilized, and its Tx-0 pin (pin 1) is connected to the Rx pin (pin D9) on the ESP8266. On the other hand, for data reception, the serial data-in on the Arduino Mega is used, and its Rx-0 pin (pin 0) is connected to the Tx pin (pin D10) on the ESP8266.

CHAPTER 4

SpO2 and Heart Rate Determination

After the above proposed architecture, we get the following outputs from the circuitry which become the input to the Arduino Mega at 4 analog input pins. These four inputs are AC and DC components of both IR and Red lights from the Nellcor-Module and the Transimpedance amplifier circuitry. The filtered and separated components look like:

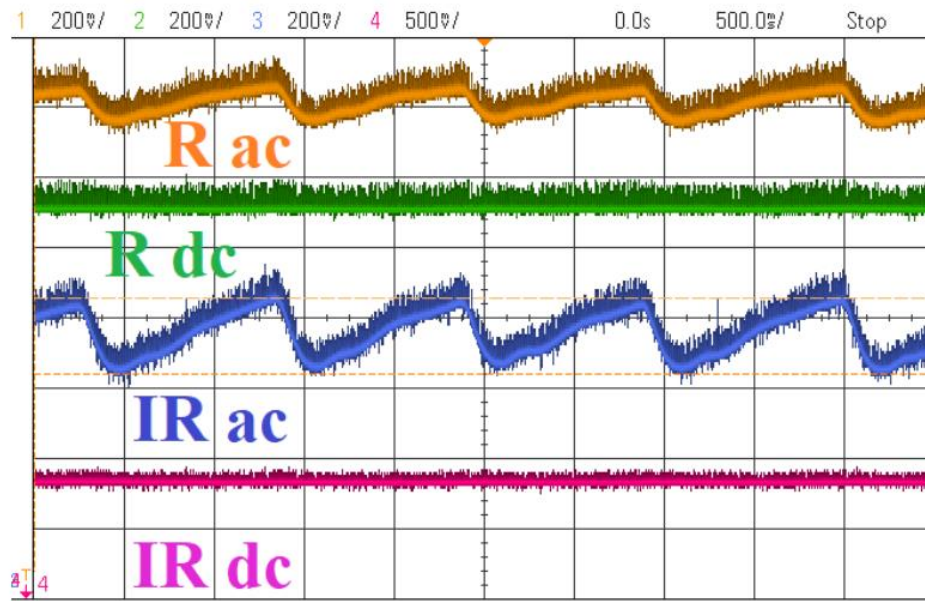


Figure 4. 1 Output of Transimpedance amplifier circuitry and input to Arduino Mega

There are two methods to find R value which in turn helps us calculate SpO2 as per the discussion in equation 2.19. Determination of heart rate will be discussed after that.

4.1 Peak-Valley Method

As per this method,

- AC component will be the difference between maximum and minimum voltage level for each heart-beat of the PPG wave for that component.
- DC component will be the minimum voltage level of the PPG wave for that component.[1]

$$R = \frac{\frac{(V_{max,R} - V_{min,R})}{V_{min,R}}}{\frac{(V_{max,IR} - V_{min,IR})}{V_{min,IR}}} \quad (4.1)$$

4.2 AC and DC Separation Method

As per this method,

- AC component will be the difference between maximum and minimum voltage level for each heart-beat of the output of 3rd order HPF for that component.
- DC component will be the maximum voltage level of the output of single order LPF for that component.[1]

$$R = \frac{\frac{V_{pk-pk,R}}{V_{dc,R}}}{\frac{V_{pk-pk,IR}}{V_{dc,IR}}} = \frac{\frac{(V_{max,HPF(R)} - V_{min,HPF(R)})}{V_{max,LPF(R)}}}{\frac{(V_{max,HPF(IR)} - V_{min,HPF(IR)})}{V_{max,LPF(IR)}}} \quad (4.2)$$

4.3 Heart Rate Determination

For finding the heart-rate these three steps are applied:

- Calculating the mean cut-off value based on PPG peak values throughout time.
- Counting the peaks over the cut-off and calculating the time interval between constituent peaks.
- Calculating the net frequency average across the aforementioned durations and changing the cut-off. [1]

$$Frequency = \frac{1000}{average\ duration} \text{ Hz} \quad (4.3)$$

$$Heart\ Rate = Frequency * 60bpm \quad (4.4)$$

4.4 Calculated Values and Comparisons

Peak Valley Method

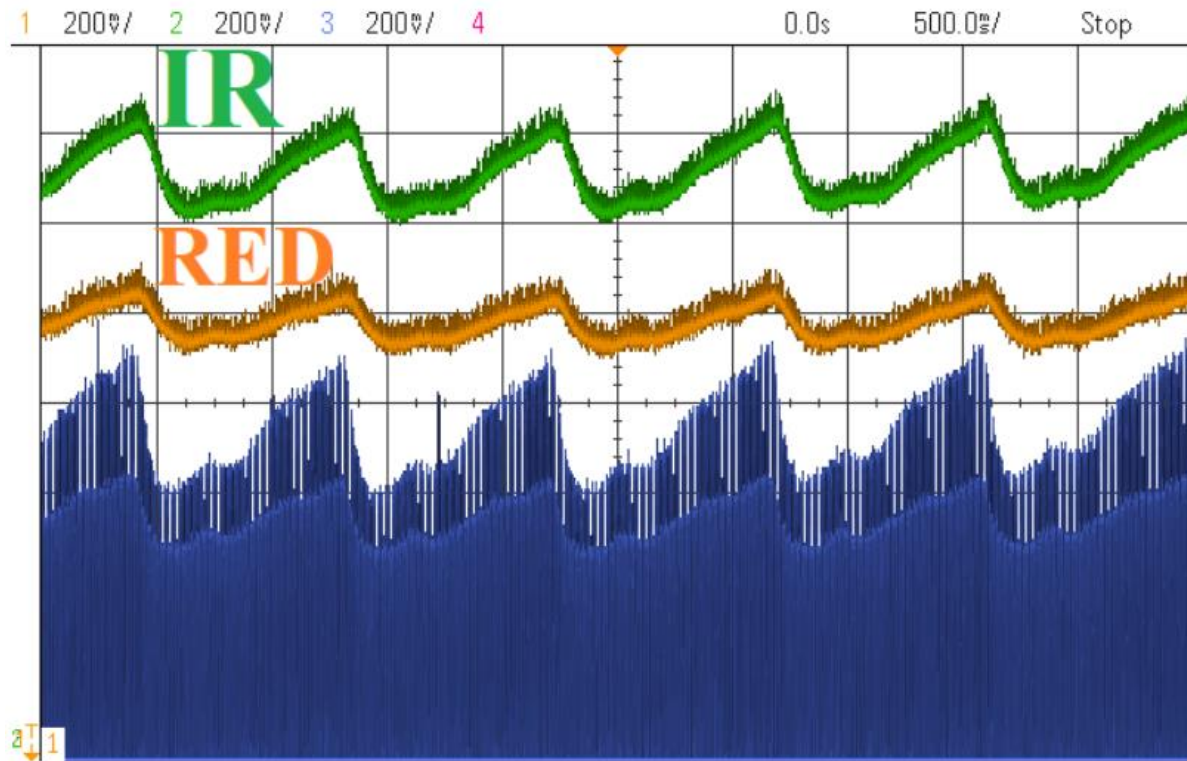


Figure 4. 2 Filtered Signals for Red and IR [11]

An example from the observation is used from Fig. 4.2 to illustrate how this method determines the SpO2. Calculations based on 4.1 are as follows:

$$V_{max, R} = 2.225V$$

$$V_{min, R} = 2.0875V$$

$$V_{max, IR} = 2.435V$$

$$V_{min, IR} = 2.21V$$

$$So, R = 0.6469$$

$$SpO2 = 98.91\%$$

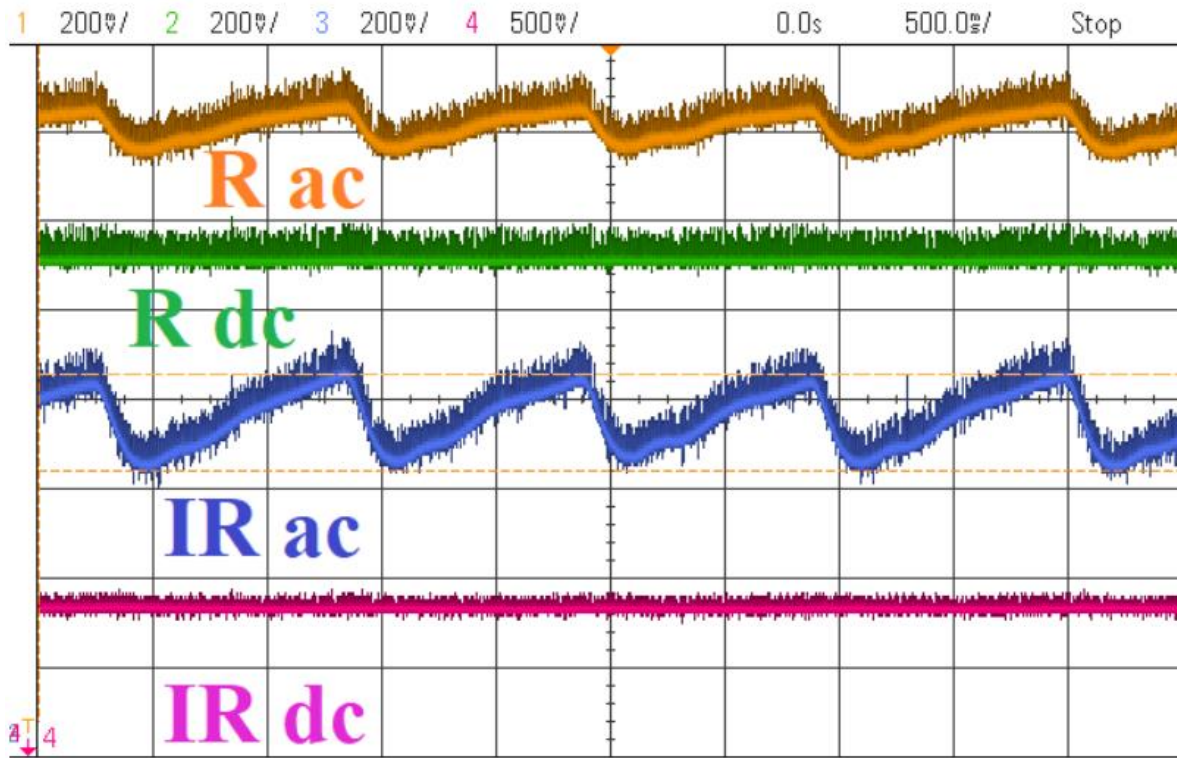
AC-DC Separation Method

Figure 4. 3 Separated AC and DC Components of Red and IR [11]

An example from the observation is used from Fig.4.3 to illustrate how this method determines the SpO2. Calculations based on 4.2 are as follows:

$$V_{pk-pk, R} = 122.5mV$$

$$V_{dc, R} = 2.000V$$

$$V_{pk-pk, IR} = 217.5mV$$

$$V_{dc, IR} = 2.206V$$

$$So, R = 0.62273$$

$$SpO2 = 99.91\%$$

Heart Rate Determination

For a value of $R=0.58$,

The frequency was found to be 0.92 Hz and thus a heart-rate of 55.28 bpm was found.

CHAPTER 5

Software Architecture

The software architecture can be divided into three parts as per the system we have chosen. They are the local Arduino/ESP setup, Firebase database setup and the REACT website setup. The local system is created to get the sensor data, process it, find the parameters required, make connection to the internet and thus send the processed data to the Firebase database. The Firebase database is created in a way to store patient information (both general and real-time health parameters). The website setup is the front-end part of the IoT interface, which is created using REACT JS and displays real-time information it gathers from the website in a easily understandable and analyzable manner.

5.1 Arduino Setup

This setup consists of the following parts:

- 1) LED driver signals from Arduino Mega
- 2) Processing data gathered from Nellcor-probe and Transimpedance Amplifier setup
- 3) Sending the data to ESP-8266
- 4) Receiving data at ESP-8266
- 5) Connection to Internet and database

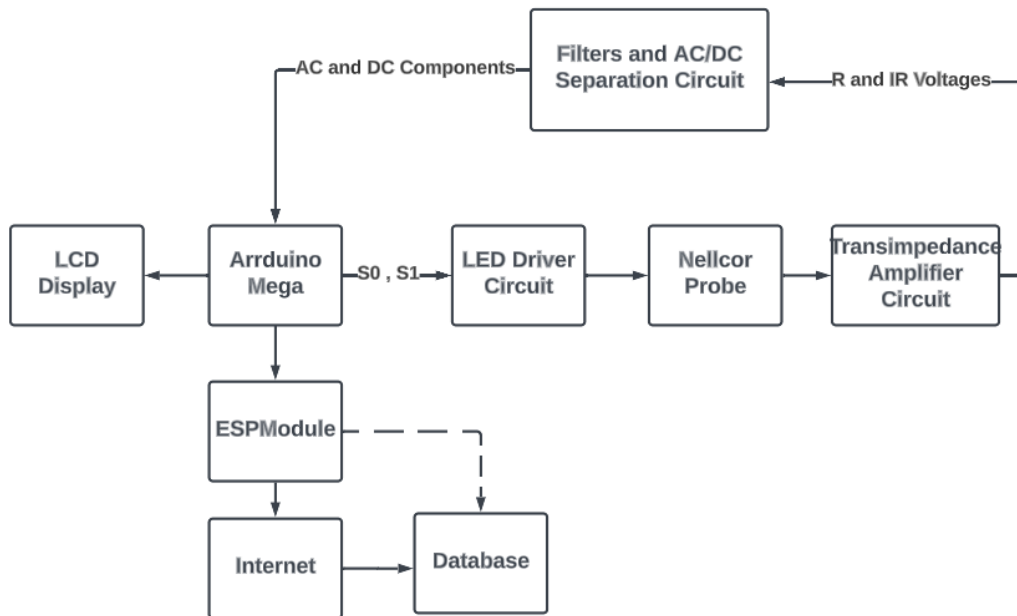


Figure 5. 1 Arduino Setup Block Diagram

LED driver signals

These signals are taken from digital pins 23 and 25. These pins are toggled 1 or 0 based on the LED we want to switch on. They can be used to configure the frequency and duty cycle of the LED signals. By keeping them ON and then creating a delay after that, they stay on for the duration of the delay and then can be kept OFF thus controlling the frequency and duty cycle of the signals. These pins 23 and 25 are the ones connected to S0 and S1 in Fig.3.6.

```
int S0 = 23; // Red driving pin
int S1 = 25; // Infrared driving pin

void setup() {
  Serial.begin(115200);           // Setting the baud Rate
  pinMode(S0, OUTPUT);           // Setting the LED Driver Pin
  pinMode(S1, OUTPUT);           // Setting the LED Driver Pin
}
```

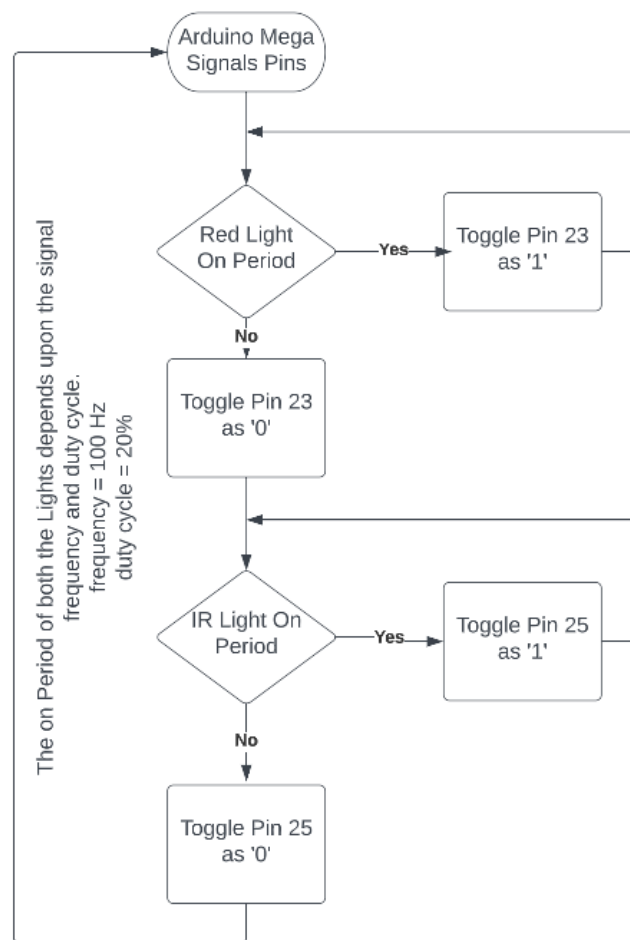


Figure 5. 2 LED driver logic flow chart

Processing data

The data that we receive are the AC and DC separated components of the Red and Infrared lights at pins A8, A9, A11, A12 of the Arduino Mega. These pins are read and the values are stored in variables as we move on to the processing.

```
float ir_value_dc = A8;
float ir_value_ac = A9;
float r_value_ac = A10;
float r_value_dc = A11;

void loop() {
// Reading and storing the values of the components
ir_ac = analogRead(ir_value_ac);
ir_dc = analogRead(ir_value_dc);
r_ac = analogRead(r_value_ac);
r_dc = analogRead(r_value_dc);
}
```

The above values are processed as required using the formulae mentioned in Chapter 2 and 4. Values are read over a range of iterations and averages are taken to generalize the calculations.

Sending the data to ESP-8266

There are two types of data that needs to be sent to the ESP Module. The first ones are the calculated SpO2, Heart Rate and Heart Rate ECG values. The second ones are the PPG waveform sample points. The PPG waveform sample points need to be sent continuously whereas the calculated values are sent after certain period of times whenever the averaging is done and the values are calculated. We create a mechanism to separate the data and distinguish them from one another as they are sent to the ESP module. The PPG waveform sample points are wrapped around small brackets, i.e., “()”. The SpO2, Heart Rate and Heart Rate ECG are wrapped around “{}”, “[]” and “^^” respectively. As we can see in Fig.5.3, the data is wrapped around respective brackets so as to distinguish them from each other. The brackets also serve as error checking parameters, i.e., if the opening bracket is missing, then it means that there is a chance the value is corrupted as it may miss some bits in the front. Thus, a value like “(699)” is processed to be 699 whereas, a value like “99)” is neglected because it will corrupt the PPG waveform and the health indicators. The data is sent by using the **Serial.print()** command which transmits the above information to serial pins of the Arduino Mega.

```
(700)
(702)
(702)
(700)
(695)
(690)
(688)
{114.28}
[2.40]
^2.77^
(686)
(696)
```

Figure 5. 3 Data being sent to the ESP Module from Arduino Mega

Receiving the data at ESP-8266

The data as shown above is received at the Serial port of ESP-module. This data is now checked to see if it is the PPG waveform sample point or the other health indicators based on the brackets they are enclosed within. They are extracted respectively into their variables if “*Serial.available()*” which means that there is something data incoming in the Serial port of the ESP module. We then read the data and process it such that every line read is cut down to a format of data enclosed within their distinguishing brackets. The SpO2, Heart Rate and Heart Rate ECG values don’t need to be uploaded at very high speed to the database whereas the PPG waveform sample points need to be uploaded at high speed so as to get a readable waveform on the website, otherwise the sampling rate would be too low for it to be considered usable.

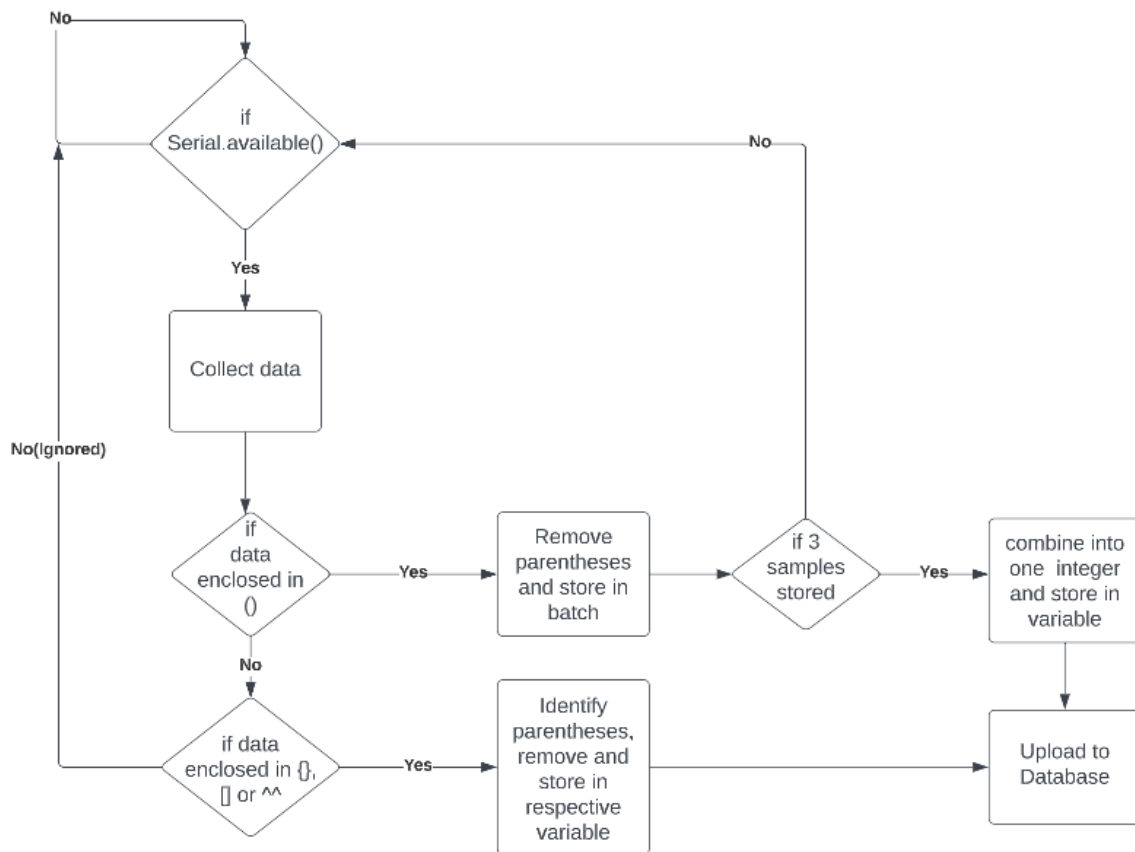


Figure 5. 4 Receiving, Processing and Sending data within ESP Module

Connection to the Internet and Database

The following libraries are included in the ESP-module code which help us connect to the internet and the Firebase database. They comprise the methods and functions that help in establishing connections and perform various operations on the connection, or the data being sent.

```
#include <ESP8266WiFi.h>           // Wi-Fi Library
#include <FirebaseESP32.h>         // Firebase Library
```

The following terms are defined as per the properties we get from the internet connection as well as the Firebase database. These will help us setup the connections.

```
#define FIREBASE_HOST "xxxxxxxxxx "
#define FIREBASE_AUTH "xxxxxxxxxx "
#define WIFI_SSID "xxxxxxxxxx"
#define WIFI_PASSWORD "xxxxxxxxxx"
```

The baud rate is set and Wi-Fi and Firebase connections are established.

```
void setup() {  
  Serial.begin(115200);  
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);  
  Firebase.begin(FIREBASE_HOST, FIREBASE_AUTH);  
}
```

Methods like `set` and `push` in the Firebase library are then used to send the values. The methods used and the techniques applied to send the values will be discussed in upcoming sections.

5.2 Database Setup

Firebase is a cloud-based platform that is incredibly adaptable and scalable and provides a variety of backend services, such as real-time databases, authentication, storage, and more. Its real-time database service is the best option for high-traffic applications because it can manage up to 100,000 simultaneous connections and 1,500 writes per second [12]. Developers may store and retrieve data in real-time, with minimal latency and high availability, using Firebase's NoSQL data model. Moreover, Firebase provides a flexible price structure that can support both small and large apps, making it an affordable choice for both new and established companies. Firebase has emerged as one of the most widely used options for contemporary app development thanks to its simplicity of use, extensive functionality, and strong community support.

The highly scalable Firebase Realtime Database is a NoSQL database with strong querying features that let developers obtain particular data in real-time. Many data types, including texts, numbers, and sophisticated data structures like maps and arrays, are supported. A further feature of Firebase Realtime Database is its adaptable security policies, which let developers manage who has access to their data. Firebase Realtime Database offers a customizable solution for real-time data storage and synchronization that can greatly cut down on development time and effort because to its simplicity of use and rich feature set.

An account had to initially be made on the Firebase website in order to construct a Firebase Realtime Database for a web application. A project was started by providing a name and choosing the relevant server location (**Singapore**) after the email address had been verified. Next, the "Realtime Database" option was chosen from the Firebase console. The database rules were declared in the **Rules** section and it was published. The use of the database was specified for a web-app. The required collections were then created, each named after a

patient's ID and storing data like name, age, gender, image URL, health indicators, and PPG waveform sample points. The database looked like the following after creating collections with fields.

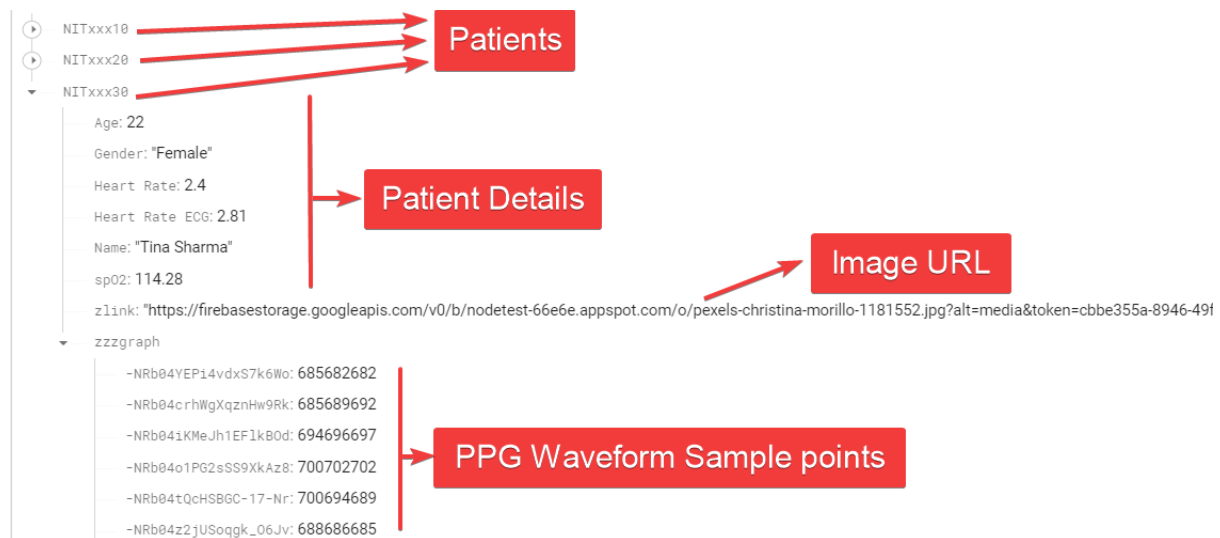


Figure 5. 5 Realtime database overview

Firestore host and auth keys were extracted from **Project Settings** after creating and setting up the database. The database SDK configuration was also taken. This information was used in the sections 5.2 and 5.4 to connect the database to the Arduino Setup and Website Setup respectively.

5.3 Website Setup

A well-known open-source JavaScript package called ReactJS is used to create dynamic user interfaces. Developers favor it because it can provide reusable UI components, which helps lower the amount of code required to create a website. React was the most widely used web framework among developers in 2021, according to a Stack Overflow study, with over 64% of developers indicating that they use it on a regular basis.

Also, ReactJS has particular characteristics that make it a good choice for real-time programming. One of these features is its virtual DOM, which enables effective real-time UI modification without the need to reload the page [13]. This is especially crucial for applications like real-time chat or stock trading systems that demand frequent updates. WebSockets, a crucial technology enabling in-the-moment communication between a server and a client, are also supported by React. This makes it feasible to construct real-time apps with the least amount of lag since it enables bidirectional, low-latency communication between the server and client.

Node.js is a strong and well-liked server-side JavaScript runtime environment that enables programmers to create web applications that are scalable and have excellent performance. For developing real-time, data-intensive applications that need quick and effective data processing, it offers an event-driven, non-blocking I/O model.

Creating the Website

There were multiple processes used with Node.js to build the Oximeter website using React. By typing the command ***“npx create-react-app”*** Oximeter into the terminal, the project was first started. The project's fundamental file structure was generated by this command. The next step was to create the three pages Home, Statistics, and Create-Patient inside the ***“src”*** folder, each of which was defined as a React component in a separate JavaScript file. React Router was used to handle page navigation; ***“the react-router-dom”*** package was installed, and a Router component was defined in the App.js file. The ***“Statistics”*** page used data from a database to provide the current health status of all registered patients, whereas the ***“Home”*** page was intended to be the website's main page. The form to add a new patient and upload their data to the database could be found on the ***“Create-Patient”*** page. The website was stylized using CSS, with individual CSS files for each page and global styles kept in a separate file called ***“App.css”***.

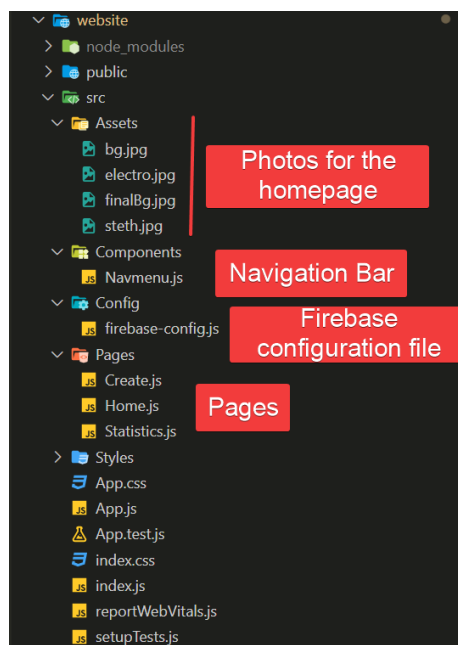


Figure 5. 6 React project overview

Connection to the Database

The Firebase SDK has to have been installed in the React project using the “*npm*” package manager in order to connect a Firebase Realtime Database to a React website. A `firebase-config.js` file was made to export the database object when firebase was installed. The configuration object retrieved from the Firebase console was also placed in this file, along with the initialization code for the Firebase SDK. The Realtime Database can be accessed via the “*db*” object, which was also used to read and write to the data kept in the database. The `firebase-config.js` file was imported into the component using the import statement in order to use the “*db*” object in a React component. This made it possible for the component to use the “*db*” object and carry out any necessary database actions. We were able to take advantage of the Firebase SDK's robust features and capabilities by integrating it into their React projects by following these procedures.

PPG Waveform Display

React ApexCharts is well-liked library for showing graphs and charts in React applications. Realtime line charts are one of ApexCharts' most notable features; they can be especially helpful in applications like finance or healthcare where the data is continually changing. The Realtime line chart gives consumers access to up-to-date data by automatically updating as new data is uploaded. ApexCharts is a flexible solution for data visualization because it provides a variety of other chart kinds, such as bar charts, pie charts, and scatter plots. One of ApexCharts' unique features is how simple and clear the API is to use, allowing programmers to easily alter the appearance and feel of their charts. A variety of color palettes, as well as the option to annotate and mark up charts, are among the customization choices offered by the library. The fact that ApexCharts is responsive and lightweight also guarantees that charts load quickly and look beautiful on a variety of devices. All things considered, React ApexCharts is a strong and adaptable alternative for showing charts and graphs in React applications, with a variety of features that make it a well-liked option among developers. This library was used to display the PPG waveform in real-time on the website.

New Patient Creation

In the Create Patient page, we created a form that let us input user details including the id, name, age, gender, health conditions and profile image. This when submitted added the user to the database, which could then be paired in the ESP-8266 module to update the health indicators to the id of the patient and thus monitor his/her status on the website.

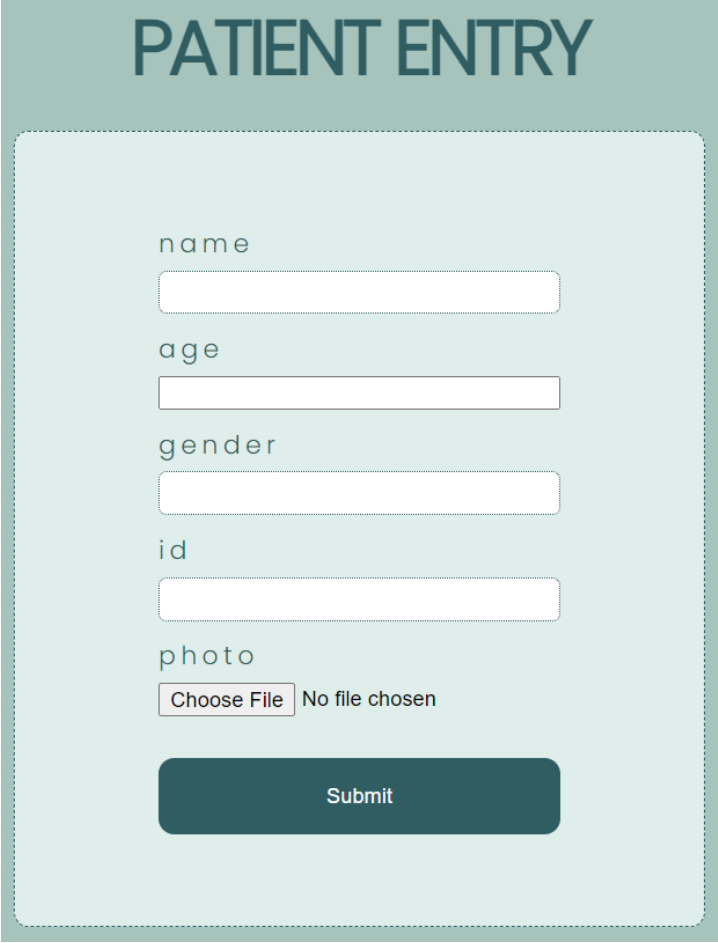
A patient entry form titled "PATIENT ENTRY" in a large, bold, dark teal font. The form is enclosed in a light teal rounded rectangle with a dashed border. It contains five input fields: "name", "age", "gender", and "id", each with a white input box. Below these is a "photo" section with a "Choose File" button and the text "No file chosen". At the bottom is a dark teal "Submit" button.

Figure 5. 7 Patient Entry Form

5.4 Code Optimization for Sampling Rates

A healthy adult male's PPG signal normally lasts between 0.6 and 1.2 seconds per cycle, depending on parameters like age, level of fitness, and environmental circumstances. So, if we assume the period to be 1 second per cycle. This means that the frequency is 1 cycle per second basically 1 Hz. As per the Nyquist's criteria for sampling rate, to represent the waveform correctly, we need at least a sampling rate of 2 times the signal frequency, which is 2 samples per second. But PPG being a sensitive health parameter would require much more samples than that to be represented correctly. We can look at a sampling rate up to 100 samples per second for prefect waveform, but we are limited by issues like network latency, internet speed, database server locations, database round trip time, hardware limitations etc. Without any optimization, there was a lot of errors in the data being sent and read by the ESP module thus, not providing an accurate representation of the PPG waveform. Some of the effects of the above parameters are:

Network latency

The lag or delay caused by the communication network when sending data between the ESP module and the Firebase server is referred to as network latency. High network latency can cause delays and erratic data update rates by slowing down the data transmission process. The distance between the ESP module and the Firebase server, the type of network infrastructure being used, and the level of network congestion can all affect the network latency. According to Google, requests sent from the United States to Firebase servers located in the United States have an average round trip time (RTT) of between 50 and 100 milliseconds.

Internet speed

The highest rate at which data may be sent between the ESP module and the Firebase server over an internet connection is referred to as internet speed. The quantity of data that can be communicated in a given amount of time might be limited by a sluggish internet connection, which can cause slower update rates and delays. The type of internet connection being utilized (such as Wi-Fi or cellular), the signal quality, and any network congestion can all affect how quickly a connection can be made. The greatest quantity of data that may be carried per second, or bandwidth, can be used to gauge internet speed. As an illustration, a Wi-Fi connection with a 10 Mbps bandwidth can send up to 10 megabits of data per second.

Database round trip time

The time needed to submit a request to the database server, receive a response, and process the data on the ESP module is referred to as the database round trip time. The magnitude of the data being transferred, network latency, and the amount of time the ESP module has to process the data can all affect the round-trip time. Google claims that the typical round-trip latency for requests delivered to US Firebase servers ranges from 50 to 100 milliseconds. [14]

Hardware restrictions

The ESP module's hardware restrictions may have an impact on how quickly data is transferred to the Firebase server. A less powered ESP module might not be able to process and transmit data as quickly as one with more processing power or memory. Moreover, network connectivity and the system's general stability may be impacted by hardware restrictions.

Since, internet speed and network latency are parameters that we may not have a hard control over at all times, they can fluctuate and cause delays and errors in data updating. We can thus

limit the sampling rate (the data uploading rate) up to 10 samples per second. This means sending 10 3-digit integer values with 100ms delay between each to the firebase server. Only 10 samples per second does sound less, but with the hardware limitations of ESP module and the free version of the database located in Singapore, achieving 10 samples per second is considerable. Some methods used to optimize the firebase portion so as to increase the sampling rates are:

Uploading Only Integers

Compared to pushing texts or arrays, pushing integers and floats from an ESP8266 module can have a number of benefits. Numerical data is better represented by integers and floats because they use less network capacity, convey data more quickly, and are more precise. Integers and floats are frequently simpler to manipulate than strings or arrays, therefore storing numerical data in Firebase may be more effective, leading to quicker data retrieval and less expensive storage. The data transmission method can be improved by pushing integers and floats to Firebase, creating a real-time web application that is more effective and quicker to respond.

Using Asynchronous Methods

For real-time applications, asynchronous Firebase methods in the ESP8266 library are typically preferable to synchronous ones. Asynchronous methods enable the ESP8266 module to carry on running code while awaiting a response from the Firebase server, which can result in code that is more responsive and efficient. Asynchronous techniques, on the other hand, pause code execution until a response is received, which can result in slower and less effective programming. “*setIntAsync()*”, which asynchronously sets an integer value in Firebase, is an example of an asynchronous Firebase method. Another illustration is “*pushIntAsync()*”, which also asynchronously pushes an integer value to a Firebase database node as a new child. These techniques allow the ESP8266 module to keep running programs while it waits for responses from the Firebase server, resulting in quicker response times and more effective programs. When employing asynchronous methods, it's also crucial to take the round-trip time (RTT) into account. The RTT is the amount of time it takes between sending a request from the ESP8266 module to the Firebase server and receiving the server's response. The effectiveness and responsiveness of the real-time application can be further increased by reducing the RTT, and asynchronous methods reduce RTT by ignoring the response time from the database.

Batch Uploading to Database

Since, uploading every single waveform sample point as soon as it arrives to the database, means we need to upload data to the servers every 100ms, this can overload the ESP-module and thus decrease its performance and cause errors and decrease sampling rate. Thus, we can use a certain mechanism to combine 3 sample points into one integer and upload to firebase every 300ms. We are hence uploading every 300ms, but still 3 samples per 300ms equals to 1 sample per 100ms, and we can save the sampling rate of 10 samples/s.

```
if (data.startsWith("(") && data.endsWith("))") {
    String graphString = data.substring(1, data.length() - 1);
    int graphVal = graphString.toInt();

    values[numValues++] = graphVal;

    if (numValues == NUM_VALUES) {
        int combined = values[0] * 1000000 + values[1] * 1000 + values[2];
        int signedValue = (int)combined;
        Firebase.RTDB.pushIntAsync(&firebaseData, id + "zzzgraph",
signedValue);
        numValues = 0;
        count++;
        memset(values, 0, sizeof(values));
    }
}
```

The above algorithm performs the batch uploading, where the NUM_VALUES value is set as 3 and thus 3 sample points are combined into one integer and uploaded.

Not Overloading Database

If the database is too cluttered, or there is a lot of complexity in the database, it can increase RTT and thus affecting the sampling rate or creating errors. Hence, we can create a mechanism to clear up the sample points once 1000 or 500 sample points are uploaded and reiterate the waveform. This can be done using the following code:

```
if (count > 500) {
    Firebase.RTDB.setAsync(&firebaseData, id + "zzzgraph", NULL);
    count = 0;
}
```

Using the above methods, in an internet speed of 10kbps with enough packet loss and network latency, we can still achieve a sampling rate of 10 samples/s.

5.5 Program Flow

Fig.5.8 below flowchart represents the flow of data and from device to device:

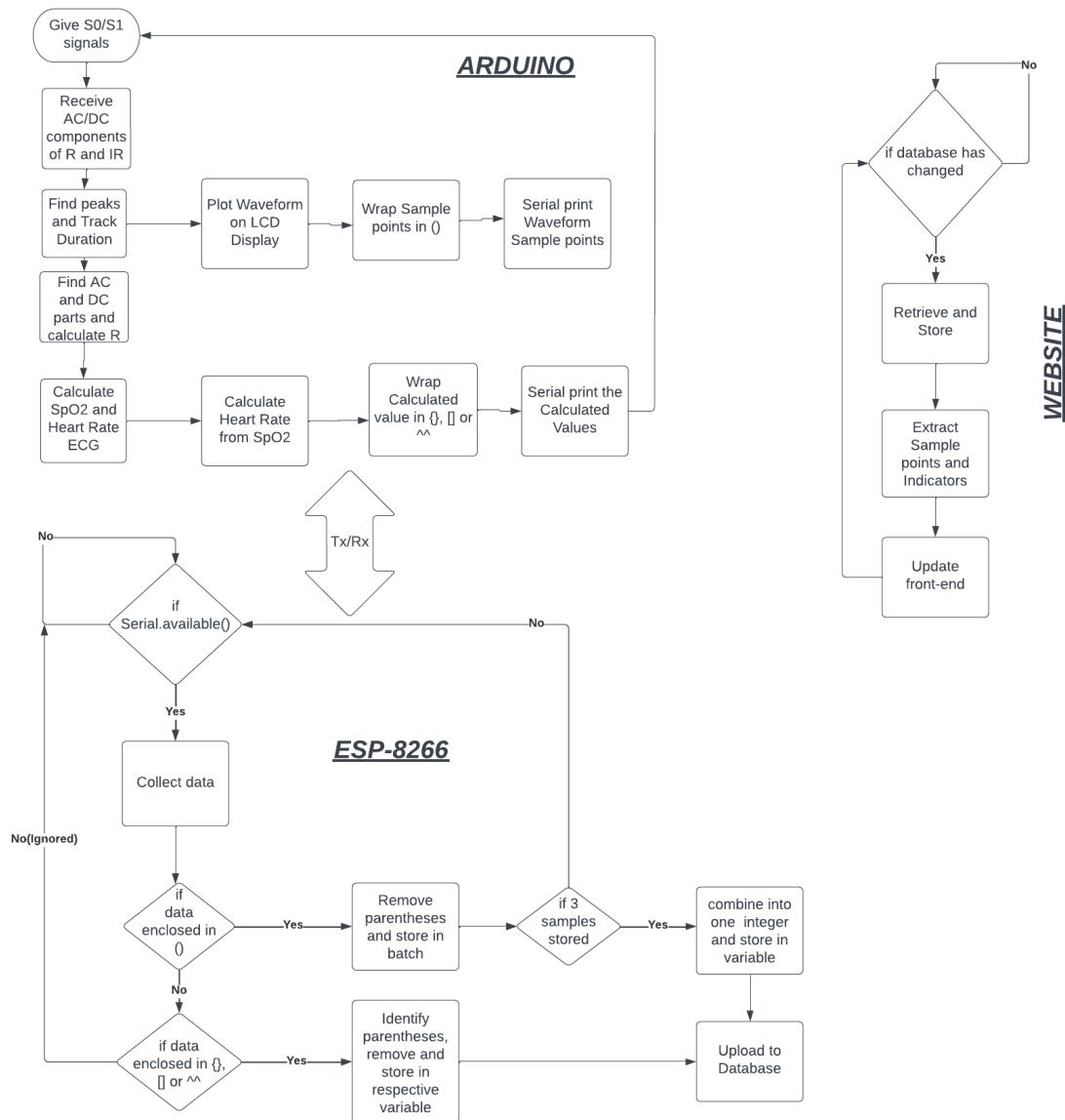


Figure 5. 8 Program and Data flow

CHAPTER 6

Results and Conclusion

6.1 Results

As discussed in Chapter 4, we were getting results comparable to a commercial PPG and SpO₂ measurement device at a local level. We displayed the information obtained on a local LCD display. We then created an IoT interface using ReactJS for the front-end, NodeJS for the backend and Firebase as the database. We achieved a sampling rate of 10 samples/second which maintained the general shape of the PPG waveform and did not induce any errors in the calculation of the SpO₂, Heart Rate and Heart Rate ECG values. We created an interface on the front-end to visualize the graph, health parameters and other patient details and compared the graphs and values locally and on the website to find that they were synced with a very less delay. A small delay at bursts was observed due to the batch upload mechanism but it was acceptable for the fact that the mechanism help improve the overall sampling rate of the waveform.

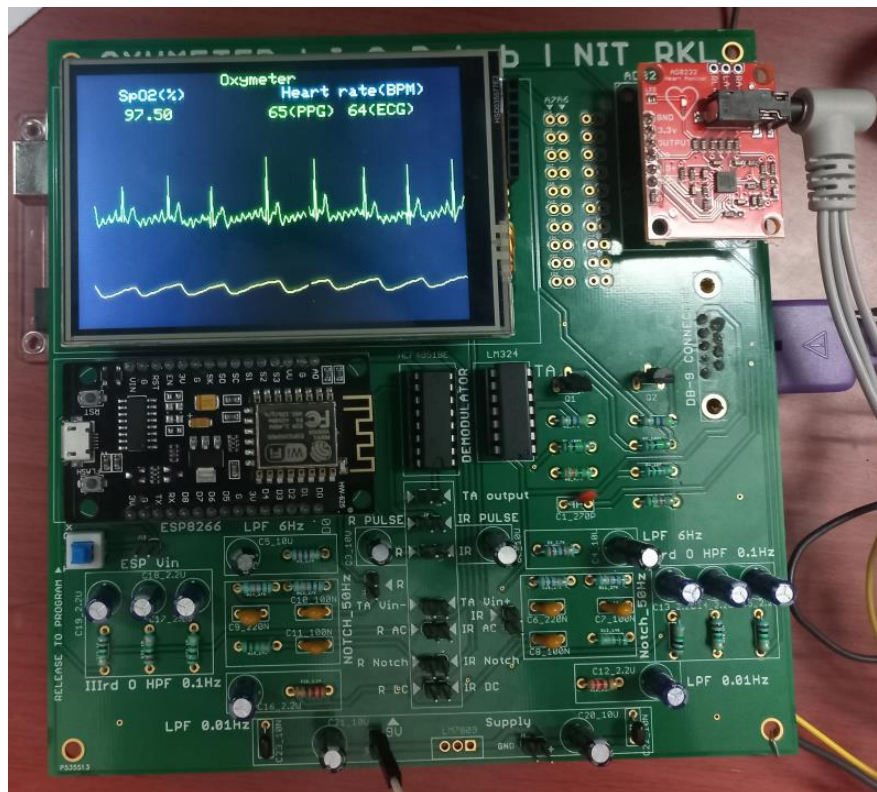


Figure 6. 1 PCB Prototype of the System [11]

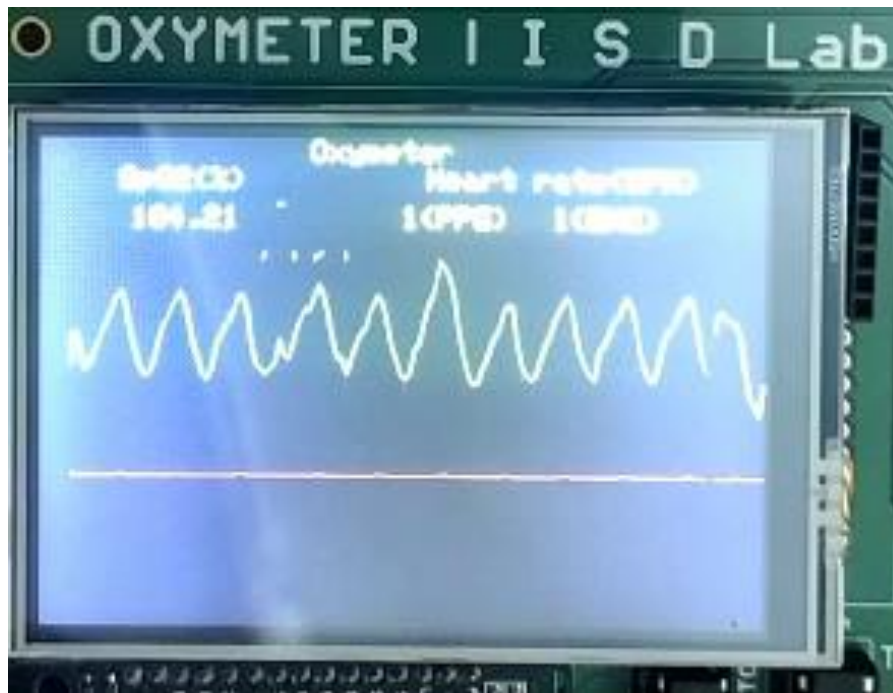


Figure 6. 2 Graph and Health Indicators on local display

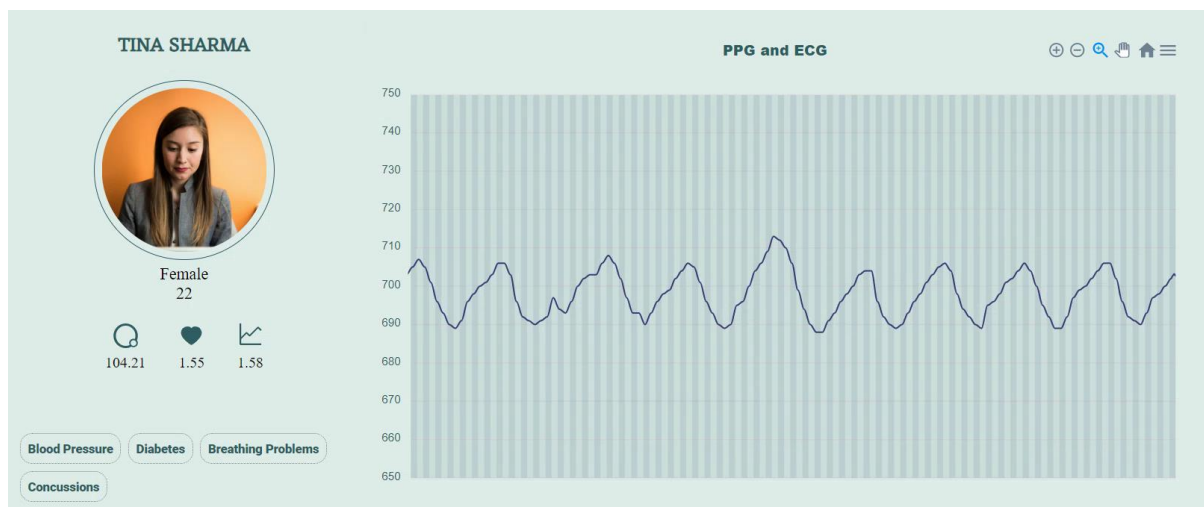


Figure 6. 3 Website UI for graph and patient details

6.2 Conclusion

The governing equations for calculating SpO₂ are derived, and the working principle of a contemporary pulse oximeter device is described. The suggested and demonstrated system design enables the system to continually measure SpO₂ and heart rate. Both PPG and ECG waves are used to monitor heart rates, and two separate methods—peak valley and AC/DC separation—are used to determine the SpO₂. The readings are shown on a TFT LCD, and through IoT connectivity, they may also be accessed on a mobile or desktop interface. React, Node, and Firebase are also used to build a fully functional website with a system for adding new users and an interface for seeing the PPG waveform and health status of registered patients in real-time. SpO₂ precision needs to be increased in the future, and the design is moved from a breadboard array to a prototype PCB. Also, hardware optimizations and usage or other databases with better servers and response times with more optimized batch uploading can be done to increase the sampling rate of the waveform being transferred to the website.

6.3 Scope for future works

In the future, current mode logic can be used in place of voltage mode to implement pulse oximetry. This strategy would result in longer battery life, more dependability, and superior calibration that is less prone to motion artefacts. Moreover, the utilization of current mode could result in low power consumption. For reliable findings, it is crucial to quickly, continuously, and precisely calibrate the voltage mode pulse oximetry device. Furthermore, in the future, an application-specific integrated circuit (ASIC) may incorporate both the voltage mode method and the current mode strategy.

Moreover, hardware related to internet connectivity can be improved, along with better internet connection with higher speed and lower latency. Usage of more real-time oriented servers with premium features and better locations can be made for better uploading of data. Better batch uploading algorithms can be discovered to transmit more data accurately without overloading the internet module.

References

- [1] J. G. Webster, Design of pulse oximeters. CRC Press, 1997.
- [2] J. Squire, "Instrument for measuring quantity of blood and its degree of oxygenation in web of the hand," Clin Sci, vol. 4, pp. 331–339, 1940.
- [3] G. A. Millikan, "The oximeter, an instrument for measuring continuously the oxygen saturation of arterial blood in man," Review of scientific Instruments, vol. 13, no. 10, pp. 434–444, 1942.
- [4] O. Glasser, "Medical physics: Vol. ii," Academic Medicine, vol. 25, no. 4, p. 302, 1950.
- [5] T. Aoyagi, "Pulse oximetry: its invention, theory, and future," Journal of anesthesia, vol. 17, no. 4, pp. 259–266, 2003.
- [6] M. Tavakoli, L. Turicchia, and R. Sarpeshkar, "An ultra-low-power pulse oximeter implemented with an energy-efficient transimpedance amplifier," IEEE transactions on biomedical circuits and systems, vol. 4, no. 1, pp. 27–38, 2009.
- [7] K. N. Glaros and E. M. Drakakis, "A sub-mw fully-integrated pulse oximeter front-end," IEEE transactions on biomedical circuits and systems, vol. 7, no. 3, pp. 363–375, 2012.
- [8] A. Azhari, S. Yoshimoto, T. Nezu, H. Iida, H. Ota, Y. Noda, T. Araki, T. Uemura, T. Sekitani, and K. Morii, "A patch-type wireless forehead pulse oximeter for spo2 measurement," in 2017 IEEE Biomedical Circuits and Systems Conference (BioCAS). IEEE, 2017, pp. 1–4.
- [9] A. Chatterjee, S. C. Kolay, M. Bandyopadhyay, and S. Chattopadhyay, "IoT based pulse oximeter system," in 2021 International Conference on Industrial Electronics Research and Applications (ICIARA). IEEE, 2021, pp. 1–4.
- [10] Identifying Appropriate Feature to Distinguish between Resting and Active Condition from FNIRS – Scientific Figure on ResearchGate. Available from: https://www.researchgate.net/figure/Absorption-coefficient-of-Hb-HbO2-and-H2O-2_fig1_296959809
- [11] A. Singh, S. K. Kar and V. K. Sinha, "Real-time Voltage Mode Pulse Oximetry System," 2022 IEEE 19th India Council International Conference (INDICON), Kochi, India, 2022, pp. 1-6, doi: 10.1109/INDICON56171.2022.10039890.
- [12] Google. (n.d.). Firebase Documentation. Retrieved May 7, 2023, from <https://firebase.google.com/docs>
- [13] ReactJS. (n.d.). Getting Started. Retrieved May 7, 2023, from <https://legacy.reactjs.org/docs/getting-started.html>
- [14] Schreiber, D. (2018, June 26). Firebase Performance: Firestore and Realtime Database Latency. Retrieved May 7, 2023, from <https://medium.com/@d8schreiber/firebase-performance-firestore-and-realtime-database-latency-13effcade26d>